MODULE 6 · SW DESIGN

Secure Coding

Most breaches start in ordinary code. Secure coding is defensive design.

Akın Kaldıroğlu · Lead Trainer

WHY IT MATTERS

Security Is Everyone's Job

Security isn't a team down the hall — it's a property of the code you write today. The cheapest bug to fix is the one you never write.

Never trust input

All external data is guilty until validated.

Fail securely

Errors must not leak secrets or open doors.

Least privilege

Give every part only the access it needs.

This is defensive programming with an adversary: the same barricade — but now the bad input is deliberate, and someone is probing for the gap you left.

WHAT YOU'LL LEAVE WITH

Learning Objectives

- Validate and sanitize every input at the boundary.
- Stop injection with parameterized queries and output encoding.
- Handle errors without leaking information — fail securely.
- Manage resources and secrets safely; apply defense in depth.

SECTION 01

Validate All Input

Every input is an attack until proven otherwise.

INPUT

Trust Nothing From Outside

Requests, files, headers, environment, other services — all of it is untrusted. Validate at the boundary, and prefer allow-lists.

Weak

- Block a few 'bad' characters (deny-list)
- Validate only on the client
- Assume the format is correct

Strong

- Allow only known-good input (allow-list)
- Validate on the server, always
- Check type, length, range, format

INPUT

Where Does Validation Live?

‘Validate at the boundary’ is only half the answer. The boundary checks the shape; the domain decides what is meaningful.

Edge — controllers, DTOs, parsers

Syntactic: is it well-formed? Type, length, format, encoding. Reject early, reject cheaply — a bad request never reaches the model.

Domain — entities & value objects

Semantic: is it meaningful? The invariants that must hold for the concept to exist at all. Enforced in the constructor, so an invalid object is unbuildable.

Everywhere else — the smell

The same if-checks repeated in service after service, because the value is still a String or a double. Miss one caller and the hole is open.

Edge validates the shape; the domain owns the meaning — then the rest of the code can trust its types.

📖 **Secure by Design — Deogun, Johnsson & Sawano** — *domain primitives: security as a design property, not bolted-on checks*

INPUT

A Value Object Validates Itself

Money is not a number and a string — it is one concept with rules. Put the rules where the concept lives:

Java *the constructor is the only door in — no invalid Money can be built*

```
public final class Money {
  private final BigDecimal amount;
  private final Currency currency;

  public static Money of(String amount, String code) {
    Currency c = Currency.getInstance(code);   // ISO 4217 allow-list, not a regex
    BigDecimal v = new BigDecimal(amount);     // never double — rounding is a bug
    if (v.signum() < 0) throw new InvalidMoney("must not be negative");
    if (v.scale() > c.getDefaultFractionDigits())
      throw new InvalidMoney("too many decimals for " + code);
    return new Money(v, c);                    // valid, or it does not exist
  }
}
```

Currency validates against ISO 4217; amount validates against that currency's own scale.

INPUT

Invariants Travel With the Type

primitives — check everywhere

```
void transfer(String from, String to,
              double amount, String ccy) {
  if (amount <= 0)      throw ...;
  if (ccy.length() != 3) throw ...;
  // …and again in every other method
  //    that touches money
}
```

value object — check once

```
void transfer(Account from, Account to,
              Money amount) {
  from.withdraw(amount);   // nothing to
  to.deposit(amount);      // validate here
}

Money add(Money other) {        // invariant
  if (!currency.equals(other.currency))
    throw new CurrencyMismatch();  // preserved
```

Validation becomes a type, not a habit. Every method that takes Money is safe by signature — and operations keep the invariant: you cannot add euros to dollars by accident.

INPUT

▶ runnable dev.kaldiroglu.bootcamp.secure.sqlinjection

Never Build Queries by Concatenation

String-building puts the user in control of your SQL. Parameterize — always:

SQL injection

```
// Java — user controls the query!
"SELECT * FROM u WHERE name='"
   + name + "'";
// name = ' OR '1'='1  → dumps table
```

Parameterized

```
// Java — driver treats name as data
var ps = con.prepareStatement(
  "SELECT * FROM u WHERE name = ?");
ps.setString(1, name);
```

Same rule in every language: prepared statements / parameters (C# SqlParameter, Python DB-API params). Never concatenate.

INPUT

Sanitize Output, Too

Validation guards what comes in; encoding guards what goes out. Data safe in a database can still be dangerous when rendered.

- Encode for the destination: HTML, URL, SQL, shell — each differs.
- Cross-site scripting (XSS): unencoded user text becomes executable HTML.
- Use framework escaping; don't hand-roll it.

SECTION 02

Fail Securely

How your code fails is part of its attack surface.

▶ runnable dev.kaldiroglu.bootcamp.secure.errorleak

Don't Leak in Error Messages

Leaks internals

```
catch (SQLException e) {
  return e.getMessage()
    + stackTrace(e);   // to the user!
  // reveals schema, paths, versions
}
```

Safe

```
catch (SQLException e) {
  log.error("query failed", e); // server
  return "Something went wrong"; // user
  // detail stays internal
}
```

Log the detail where only you can see it; show the user a generic, honest message.

ERRORS

Exceptions, Not Silent Failure

Dangerous

- Empty catch blocks that swallow errors
- Returning null / -1 to signal failure
- Continuing in an invalid state

Safe

- Throw meaningful, specific exceptions
- Fail fast — stop before damage spreads
- Leave the system in a valid state

SECTION 03

Resources & Secrets

Clean up what you open; never hard-code what must stay secret.

RESOURCES

Release What You Acquire

Leaked connections, files and memory cause outages — and outages are security incidents. Let the language guarantee cleanup.

- Use try-with-resources (Java), using (C#), with (Python) — deterministic cleanup.
- Bound everything: buffer sizes, upload limits, request timeouts.
- In manual-memory languages: validate lengths; avoid overflows and use-after-free.

RESOURCES

▶ runnable dev.kaldiroglu.bootcamp.secure.secrets

Keep Secrets Out of Code

Never

```
String key =
  "sk_live_9f8a...";  // in source!
// leaks to git, logs, screenshots
```

Instead

```
String key =
  System.getenv("API_KEY");
// config / secret manager / vault
```

Secrets belong in environment config or a secret manager — never in the repository, logs, or error messages.

THE MINDSET

Defense in Depth

No single control is enough. Layer them, so one failure doesn't become a breach.

Validate

Reject bad input at every boundary.

Least privilege

Minimal rights for each user, service, token.

Assume breach

Log, monitor, and limit the blast radius.

THE AI ANGLE

AI Can Introduce Vulnerabilities

Generated code often takes the insecure shortcut — string-built queries, swallowed errors, hard-coded keys. You are the security reviewer.

- Explicitly ask for parameterized queries and validated input.
- Review AI diffs for injection, secret leaks, and unsafe error handling.
- Never paste real secrets or production data into a prompt.

REMEMBER THESE

Key Takeaways

- ✓ Never trust input — validate at the boundary with allow-lists.
- ✓ Parameterize queries; encode output. No string-built SQL or HTML.
- ✓ Fail securely: log detail internally, show users a generic message.
- ✓ Release resources deterministically; keep secrets out of code.
- ✓ Defense in depth — and treat AI-generated code as unreviewed.

GO DEEPER

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Secure by Design** Deogun, Johnsson & Sawano · Manning, 2019

Security as a design property — value objects and invariants, not bolted-on checks.

📖 **Threat Modeling: Designing for Security** Adam Shostack · Wiley, 2014

How to find what can go wrong, systematically, before writing the code.

📖 **Writing Secure Code, 2nd ed.** Howard & LeBlanc · Microsoft Press, 2003

Still the clearest treatment of input validation, least privilege and secret handling.

📖 **The Web Application Hacker's Handbook, 2nd ed.** Stuttard & Pinto · Wiley, 2011

The attacker's view — read it to understand why the defenses are shaped as they are.

OWASP Top 10 & Cheat Sheet Series — the working reference → `owasp.org`

NEXT SESSION

What's Next

MODULE 7 · TUESDAY 28 JULY

Developer Testing

with Akın Kaldıroğlu

Unit and integration testing — the safety net that lets you refactor, secure and evolve code with confidence.

FURTHER PRACTICE

Guard the Boundaries

Trace where untrusted input enters the ACME Orders system, and make it validate and fail safely.

Refactoring course · Defensive Programming

Repo `github.com/kaldiroglu/refactoring-course-student-materials`

IN THIS SESSION, LOOK AT

- `Slides-PDF/Ch08-DefensiveOffensive.pdf`
- `.../web/OrderController.java  (input boundary)`
- `.../util/Util.java  (unchecked helpers)`

Questions?

Assume nothing. Validate everything.

Akın Kaldıroğlu · Secure Coding